

S2R

Speech-to-Retrieval System

Pipeline complet de recherche documentaire par la voix

Rapport Technique — Pipeline P1 → P4

P1 Data Engineer | P2 Speech AI Engineer

P3 ML Engineer | P4 System Integration

Institut National des Postes et Telecommunications (INPT)

Rabat, Maroc — 2024-2025

Dataset

4 985 audios

Corpus

6 744 chunks

Modele

Wav2Vec2

Espace

dim = 256

Table des matières

1. Introduction

Ce rapport presente le travail realise dans le cadre du projet *Speech-to-Retrieval System (S2R) using Deep Learning*, developpe a l’Institut National des Postes et Telecommunications (INPT). Il couvre l’integralite des quatre parties du pipeline :

- **P1 — Data Engineer** : preparation du dataset speech → document
- **P2 — Speech AI Engineer** : encodage vocal sans ASR via Wav2Vec2
- **P3 — ML Engineer** : entrainement du Dual Encoder (alignement audio–texte)
- **P4 — System Integration** : API REST FastAPI, interface Gradio, deployment

Un systeme Speech-to-Retrieval (S2R) permet de retrouver des documents textuels pertinents a partir d’une requete vocale, **sans passer par une etape de transcription ASR**. L’audio est directement converti en vecteur d’embedding dans un espace semantique partage avec les documents textuels, ce qui rend le systeme robuste aux accents et aux langues mixtes.

TABLE 1 – Vue d’ensemble du pipeline S2R

Partie	Module	Role	Statut
P1	notebooks/P1_*.ipynb	Donnees audio + texte	✓ Termine
P2	src/speech_encoder.py	Encodage audio Wav2Vec2	✓ Termine
P3	training/	Dual Encoder (InfoNCE)	✓ Termine
P4	api/ + demo/	API FastAPI + Gradio	✓ Termine

1.1 Architecture globale

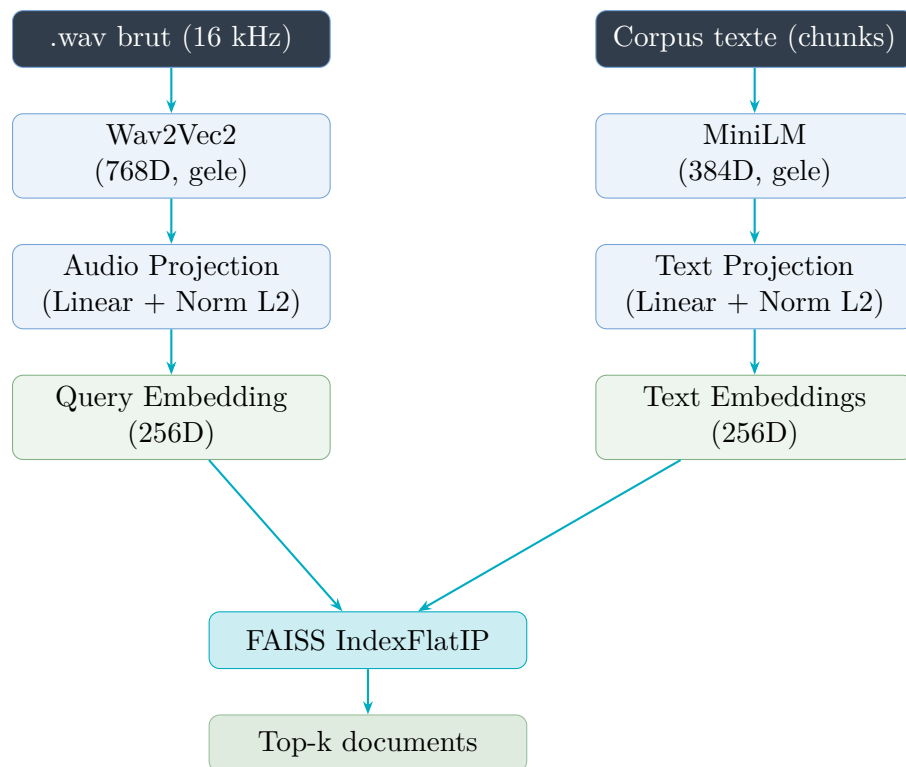


FIGURE 1 – Architecture complete du systeme Speech-to-Retrieval

2. P1 — Data Engineer

La partie P1 est responsable de la construction et de la preparation du dataset *speech* → *document*. Elle fournit l'ensemble des donnees necessaires aux parties P2 et P3.

2.1 Collecte des donnees audio

Les donnees audio ont ete collectees depuis **openslr/librispeech_asr** (HuggingFace), un dataset anglophone issu de livres audio du domaine public. Deux splits ont ete utilises :

- **train.100** — 100 heures d'audio propre, 3 000 echantillons
- **train.360** — 360 heures d'audio propre, 2 000 echantillons

TABLE 2 – Caracteristiques du dataset audio brut

Parametre	Valeur
Dataset	openslr/librispeech_asr (HuggingFace)
Langue	Anglais (English)
Splits	train.100 + train.360
Format brut	Divers sample rates (16 kHz, 22 kHz, 44 kHz)
Acces	Public — aucune authentication requise

2.2 Pretraitement audio

Chaque fichier audio a subi un pipeline de nettoyage standardise avant d'etre sauvegarde en format **WAV 16 kHz mono** :

1. **Resample** — Conversion vers 16 000 Hz via `librosa.resample()`
2. **Mono** — Conversion stereo → mono
3. **Filtre duree** — Rejet des audios < 1 s, troncature a 20 s
4. **Filtre SNR** — Rejet des audios avec SNR < 5 dB
5. **Normalisation peak** — Normalisation amplitude a $[-1, 1]$

```

1 signal_power = np.mean(waveform ** 2) + 1e-10
2 noise_floor  = np.percentile(np.abs(waveform), 10) ** 2 + 1e-10
3 snr_db = 10 * np.log10(signal_power / noise_floor)
4 if snr_db < MIN_SNR_DB: # seuil = 5 dB
5     return None # audio rejete

```

Listing 1 – Estimation du SNR (Signal-to-Noise Ratio)

Resultat : 4 985 fichiers WAV acceptes sur 5 000 cibles (99.7%). Les 15 fichiers rejetes presentaient un SNR insuffisant ou une duree trop courte.

2.3 Construction du corpus texte

Le corpus textuel a été construit à partir d'articles Wikipedia en anglais via **wikimedia/wikipedia (20231101.en)** : 600 articles téléchargés en streaming, filtrage des articles < 150 mots, **570 articles retenus**.

Chaque article a été découpé en chunks de **512 tokens** avec un overlap de **50 tokens** (fenêtre glissante) :

```
1 tokenizer = AutoTokenizer.from_pretrained(
2     "sentence-transformers/all-mpnet-base-v2"
3 )
4 tokens = tokenizer.encode(document_text,
5     add_special_tokens=False)
6 chunk_size, overlap = 512, 50
7 for start in range(0, len(tokens), chunk_size - overlap):
8     end = min(start + chunk_size, len(tokens))
9     chunk = tokenizer.decode(tokens[start:end])
```

Listing 2 – Pipeline de chunking avec fenêtre glissante

TABLE 3 – Statistiques du corpus texte

Metric	Valeur	Unité
Total chunks produits	6 744	chunks
Articles source	570	documents
Tokens moyens/chunk	493	tokens
Taille fichier CSV	15 719	KB

2.4 Creation des paires audio–document et split

Chaque fichier audio est associé au chunk texte le plus pertinent via la **similarité de Jaccard** sur les mots-clés :

$$J(A, B) = \frac{|KW(A) \cap KW(B)|}{|KW(A) \cup KW(B)|} \quad (1)$$

Score Jaccard moyen : 0.073

Score faible attendu : les transcriptions LibriSpeech sont des phrases courtes tandis que les chunks Wikipedia sont des paragraphes techniques denses. Les paires servent uniquement à l'entraînement contrastif de P3.

TABLE 4 – Repartition train / val / test (strategie 80/10/10)

Split	Paires	Proportion	Fichier
Train	3 988	80%	pairs_train.csv
Validation	498	10%	pairs_val.csv
Test	499	10%	pairs_test.csv
Total	4 985	100%	pairs.csv

3. P2 — Speech AI Engineer

La partie P2 transforme les fichiers audio en vecteurs d’embeddings denses de dimension **768**, normalisés L2, directement exploitables par le Dual Encoder de P3.

3.1 Modele Wav2Vec2

Le modele utilise est **facebook/wav2vec2-base-960h**, pre-entraine par Meta AI sur 960 heures d’audio anglais (LibriSpeech). C’est un Transformer auto-supervise qui apprend des representations speech riches sans supervision explicite.

TABLE 5 – Parametres du modele Wav2Vec2

Parametre	Valeur	Description
Modele	wav2vec2-base-960h	Version base
Parametres	94M	Total parametres
Sample rate	16 000 Hz	Format audio requis
Hidden dim	768	Dimension des hidden states
Nb couches	12	Transformer layers
Nb tetes	12	Attention heads
Poids	Geles	frozen=True pendant training
Pooling	Mean	Moyenne sur dimension temporelle

3.2 Architecture du SpeechEncoder

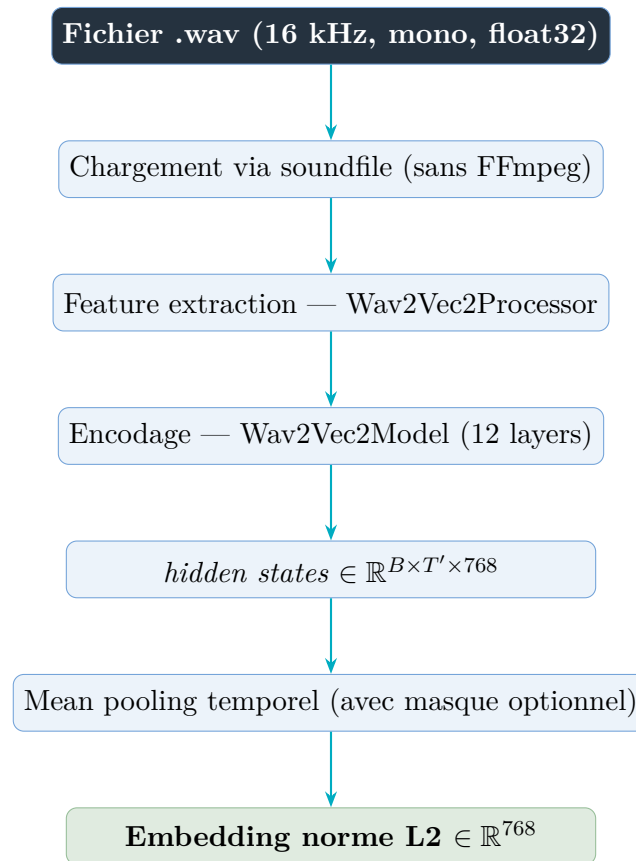


FIGURE 2 – Pipeline d’encodage audio P2

3.3 Gestion du chargement audio (soundfile)

Sur Windows, la dépendance `torchaudio.load` requiert FFmpeg via `torchcodec`, ce qui genere des erreurs sur certains environnements. La solution adoptee utilise directement `soundfile` :

```

1 import soundfile as sf
2
3 def _load_wav(audio_path, target_sr=16_000):
4     data, sr = sf.read(audio_path, dtype="float32",
5         always_2d=True)
6     waveform = torch.from_numpy(data.T) # (channels, samples)
7     if waveform.shape[0] > 1: # stereo -> mono
8         waveform = waveform.mean(dim=0, keepdim=True)
9     if sr != target_sr:
10         # torchaudio.functional.resample = op PyTorch pur, sans backend
11         waveform = torchaudio.functional.resample(waveform, sr, target_sr)
12     return waveform.squeeze(0) # [T]

```

Listing 3 – Chargement audio robuste (src/speech_encoder.py)

3.4 Mean pooling avec masque d'attention

Pour les batches paddes, un masque d'attention est propage jusqu'à la sortie du feature extractor pour un pooling correct :

$$\mathbf{e} = \frac{\sum_{t=1}^{T'} \mathbf{h}_t \cdot m_t}{\sum_{t=1}^{T'} m_t} \quad m_t \in \{0, 1\} \quad (2)$$

La normalisation L2 finale garantit $\|\mathbf{e}\|_2 = 1$, rendant la similarité cosinus équivalente au produit scalaire, ce qui est optimal pour FAISS IndexFlatIP.

$$\hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|_2} \implies \cos(\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2) = \hat{\mathbf{e}}_1^\top \hat{\mathbf{e}}_2 \quad (3)$$

3.5 Outputs de P2

TABLE 6 – Fichiers produits par P2

Fichier	Dimensions	Description
audio_embeddings.npy	(4985, 768)	Matrice des embeddings Wav2Vec2
audio_embeddings_index.csv	4985 lignes	Mapping audio_id → ligne
src/speech_encoder.py	—	Module importable par P3/P4

4. P3 — ML Engineer : Dual Encoder

La partie P3 entraine un **Dual Encoder** qui aligne l'espace audio (768D) et l'espace texte (384D) dans un espace commun partage de dimension **256D**, via une fonction de perte contrastive.

4.1 Architecture du DualEncoderModel

Le modele est compose de deux branches de projection symetriques :

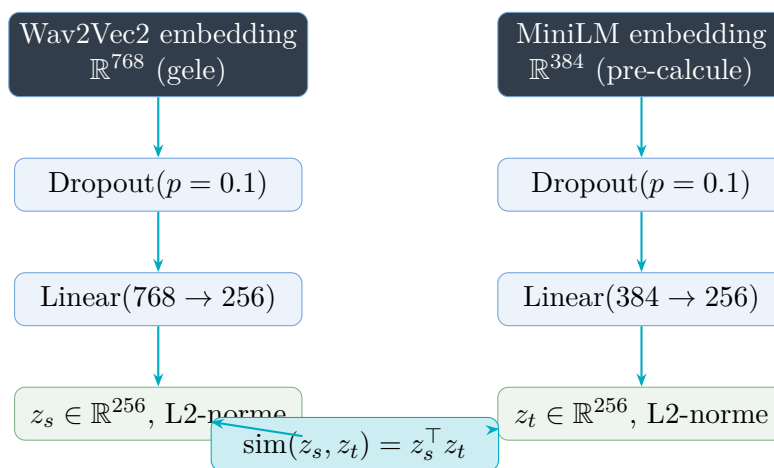


FIGURE 3 – Architecture du DualEncoderModel (mode rapide)

```

1 class DualEncoderModel(nn.Module):
2     def __init__(self, audio_input_dim=768, projection_dim=256,
3                   dropout=0.1, text_input_dim=384):
4         super().__init__()
5         self.text_encoder = None # mode rapide
6         self.audio_projection = nn.Sequential(
7             nn.Dropout(dropout),
8             nn.Linear(audio_input_dim, projection_dim),
9         )
10        self.text_projection = nn.Sequential(
11            nn.Dropout(dropout),
12            nn.Linear(text_input_dim, projection_dim),
13        )
14
15    def encode_audio(self, x):
16        return F.normalize(self.audio_projection(x), p=2,
17                           dim=-1)
18
19    def encode_text_precomputed(self, x):
20        return F.normalize(self.text_projection(x), p=2, dim=-1)
  
```

Listing 4 – DualEncoderModel simplifie (training/models.py)

4.2 Mode rapide (pre-calcul des embeddings texte)

Sur CPU, encoder MiniLM a chaque batch pendant l'entraînement serait prohibitif. La solution adoptée est le **mode rapide** :

1. Pre-calculer *une seule fois* tous les embeddings MiniLM (384D) en `embeddings/precomputed_embeddings`
2. Pendant l'entraînement, charger ces vecteurs depuis la mémoire (numpy array)
3. Uniquement les deux couches **Linear** sont entraînables

TABLE 7 – Comparaison mode complet vs mode rapide

Critere	Mode complet	Mode rapide
Forward text/batch	MiniLM complet	Lookup numpy
Temps/epoque (CPU)	~30 min	~45 secondes
Parametres entrain.	22M (MPNet)	2 couches Linear
Requiert GPU	Oui	Non
Speedup	1×	20–50×

Acceleration mesuree sur Intel Core i7, 24 Go RAM : mode complet $\approx$ 2h20/epoque $\rightarrow$ mode rapide $\approx$ 45 sec/epoque. Entrainement complet de 20 epoques ramene de $\approx$ 2 jours a $\approx$ 15 minutes.

4.3 Fonction de perte

Deux fonctions de perte sont implementees dans `training/losses.py`.

4.3.1 Contrastive Loss (InfoNCE / NT-Xent)

Pour un batch de N paires $(z_s^{(i)}, z_t^{(i)})$, la matrice de similarites est :

$$S_{ij} = \frac{z_s^{(i)} \cdot z_t^{(j)}}{\tau}, \quad \tau = 0.07 \quad (4)$$

La perte InfoNCE penalise les similarites hors-diagonale (negatifs in-batch) :

$$\mathcal{L} = \frac{1}{2} \left(\underbrace{\frac{1}{N} \sum_{i=1}^N -\log \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}}}_{\mathcal{L}_{s \rightarrow t}} + \underbrace{\frac{1}{N} \sum_{i=1}^N -\log \frac{e^{S_{ii}}}{\sum_j e^{S_{ji}}}}_{\mathcal{L}_{t \rightarrow s}} \right) \quad (5)$$

```

1 def contrastive_loss(speech_emb, text_emb, temperature=0.07):
2     logits = (speech_emb @ text_emb.T) / temperature
3     labels = torch.arange(logits.size(0), device=logits.device)
4     loss_s2t = F.cross_entropy(logits, labels)
5     loss_t2s = F.cross_entropy(logits.T, labels)
6     return (loss_s2t + loss_t2s) / 2.0

```

Listing 5 – Contrastive loss (InfoNCE) – `training/losses.py`

4.3.2 Triplet Loss

Comme alternative, une triplet loss avec hard-negative mining est disponible :

$$\mathcal{L}_{triplet} = \frac{1}{2} \left(\text{ReLU}(d^+ - d_{text}^- + m) + \text{ReLU}(d^+ - d_{speech}^- + m) \right) \quad (6)$$

avec $d^+ = 1 - \text{sim}(z_s^{(i)}, z_t^{(i)})$ et $m = 0.2$.

4.4 Procedure d'entraînement

TABLE 8 – Hyperparametres d'entraînement recommandes

Parametre	Valeur locale (CPU)	Valeur cloud (GPU T4)
Epoques	20	20
Batch size	16	64
Learning rate	2e-4	2e-4
Warmup steps	100	100
Grad clipping	1.0	1.0
Projection dim	256	256
Temperature	0.07	0.07
Optimiseur	AdamW	AdamW

Le scheduler de learning rate combine un warmup lineaire sur les premiers 100 steps puis un decroissance cosinus jusqu'à la fin de l'entraînement.

4.5 Metriques d'évaluation

Trois metriques sont calculees a chaque epoque de validation dans `training/metrics.py` :

- **Recall@k** : proportion de requetes pour lesquelles le document correct apparait dans le top- k ($k = 5$ et 10)

$$\text{Recall}@k = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\text{rank}(q) \leq k] \quad (7)$$

- **MRR** (Mean Reciprocal Rank) : moyenne des inverses du rang du premier document pertinent

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}(q)} \quad (8)$$

TABLE 9 – Objectifs de performance apres 20 epoques

Metrique	Objectif	Interpretation
Recall@5	> 0.30	30% des requetes trouvees dans le top-5
Recall@10	> 0.45	45% des requetes trouvees dans le top-10
MRR	> 0.25	rang moyen reciproque du document correct

Limitation intrinseque du dataset

Les paires audio-document sont construites par similarite Jaccard (score moy. 0.073). Les paires ne sont pas semantiquement parfaites : les metriques mesurent la capacite du modele a retrouver ces paires imparfaites, pas la qualite semantique absolue.

4.6 Export des embeddings texte (post-entrainement)

Apres l'entrainement, `training/export_text_embeddings.py` projette tous les chunks texte dans l'espace commun 256D :

1. Charger les embeddings pre-calculés (384D) depuis `precomputed_text.npy`
2. Appliquer `model.encode_text_precomputed()` par batch de 64
3. Sauvegarder la matrice ($N, 256$) dans `text_chunk_embeddings.npy`
4. Sauvegarder le manifest (`chunk_id, document_id, text`)

5. P4 — System Integration

La partie P4 expose le systeme S2R via deux interfaces : une **API REST FastAPI** pour l'integration backend, et une **interface Gradio** pour la demonstration interactive.

5.1 Index FAISS

La recherche des documents pertinents est effectuee via **FAISS IndexFlatIP** (produit scalaire exact sur vecteurs normalises = cosine similarity). L'index est construit en memoire au demarrage :

```

1 embs =
    np.load("embeddings/text_chunk_embeddings.npy").astype("float32")
2 norms = np.linalg.norm(embs, axis=1, keepdims=True)
3 embs /= np.clip(norms, 1e-9, None)           # normalisation L2
4
5 index = faiss.IndexFlatIP(embs.shape[1])      # dim = 256
6 index.add(embs)                             # N vecteurs
    indexes
7 # Recherche : scores, indices = index.search(query, k)

```

Listing 6 – Construction de l'index FAISS (api/main.py)

5.2 API REST FastAPI

L'API expose 5 endpoints documentes via Swagger (disponible sur /docs) :

TABLE 10 – Endpoints de l'API FastAPI (api/main.py)

Methode	Route	Description
GET	/health	Etat du service (dual encoder charge, taille index)
POST	/search/audio	Upload fichier .wav → top-k documents
POST	/search/base64	Audio encode base64 → top-k documents
POST	/search/text	Requete texte (MiniLM) → top-k documents
GET	/examples	Liste des fichiers audio de demonstration

```

1 {
2   "results": [
3     {"rank": 1, "score": 0.8241, "chunk_id":
4       "doc00042_chunk003",
5       "text": "Anarchism is a political philosophy..."},
6     {"rank": 2, "score": 0.7955, "chunk_id":
7       "doc00042_chunk004",
8       "text": "The anarchist movement emerged..."}
9   ],
10  "mode": "dual_encoder",
11  "query_dim": 256,

```

```

10  "index_size": 6744
11  }

```

Listing 7 – Exemple de reponse /search/audio

Lancement de l'API :

```

1  # Variables d'environnement (optionnelles)
2  export S2R_CHECKPOINT="models/dual_encoder_mpnet/best_model.pt"
3  export
4      S2R_TEXT_EMBEDDINGS="embeddings/text_chunk_embeddings.npy"
5  export S2R_MANIFEST="embeddings/text_chunk_manifest.csv"
6
7  # Demarrage
8  uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
9  # Documentation interactive : http://localhost:8000/docs

```

Listing 8 – Lancement de l'API FastAPI

5.3 Interface Gradio

L'interface Gradio (demo/app.py) propose deux onglets :

- **Onglet 1 — Recherche vocale** : upload ou enregistrement microphone, slider pour le nombre de resultats, liste deroulante d'exemples pre-charges
- **Onglet 2 — Recherche texte** : saisie textuelle directe, comparaison avec le mode audio pour valider l'index

TABLE 11 – Options de lancement de la demo Gradio

Commande	Description
python demo/app.py	Mode prototype (Wav2Vec2 brut)
python demo/app.py -checkpoint best_model.pt	Dual encoder entraine
python demo/app.py -share	Lien public Gradio (72h)
python demo/app.py -port 7861	Port personnalise

Au demarrage, la demo affiche un panneau de statut indiquant le mode actif (prototype ou dual encoder entraine), la taille de l'index FAISS, et le device utilise.

6. Pipeline d'exécution

6.1 Pipeline automatisé (run_pipeline.bat / .sh)

Deux scripts d'automatisation complets sont fournis pour Windows et Linux/macOS. Le pipeline s'exécute en 6 étapes séquentielles :

TABLE 12 – Etapes du pipeline automatisé

Etape	Module	Action
1	validate_handoff	Vérification des données P1/P2
2	precompute_text	Pre-calcul embeddings MiniLM (une fois)
3	train_dual_encoder	Entraînement 20 époques, batch 16
4	export_text_embeddings	Export embeddings 256D vers FAISS
5	inference.py	Test d'inférence sur un fichier exemple
6	API ou Demo (optionnel)	Lancement selon <code>-api</code> ou <code>-demo</code>

```

1 run_pipeline.bat                                # Pipeline complet (20
   epoques)
2 run_pipeline.bat --skip-train                    # Sauter l'entraînement
   (checkpoint present)
3 run_pipeline.bat --skip-train --demo             # Sauter + lancer Gradio
4 run_pipeline.bat --skip-train --api              # Sauter + lancer FastAPI

```

Listing 9 – Options du pipeline Windows

6.2 Entraînement sur Google Colab / Kaggle

Un notebook Jupyter complet est fourni dans `notebooks/S2R_Colab_Kaggle.ipynb` pour l'entraînement sur GPU cloud :

TABLE 13 – Performances d'entraînement selon la plateforme

Plateforme	GPU	Temps/époque	20 époques
Local (Intel i7, CPU)	Aucun	≈ 45 s	≈ 15 min
Google Colab (gratuit)	T4 15GB	≈ 60 s	≈ 20 min
Kaggle (gratuit)	T4 16GB	≈ 55 s	≈ 18 min
Google Colab Pro	A100 40GB	≈ 20 s	≈ 7 min

Le notebook couvre : installation des dépendances, chargement depuis Google Drive ou upload direct, validation des données, pre-calcul MiniLM, entraînement avec `tqdm` progress, évaluation, et téléchargement du checkpoint.

6.3 Verification des metriques

```
1 python -m training.validate_handoff \  
2     --audio-manifest-csv  embeddings/audio_embeddings_index.csv \  
3     --audio-embeddings-npy embeddings/audio_embeddings.npy \  
4     --pairs-csv           data/output/pairs_train.csv  
5  
6 # Attendu :  
7 # manifest_rows=4985  
8 # embedding_shape=(4985, 768)  
9 # status=manifest_and_embeddings_ok  
10 # pairs_rows=3988  
11 # status=pairs_alignment_ok
```

Listing 10 – Validation des donnees P1/P2 (training/validate_handoff.py)

7. Integration Snowflake

7.1 Architecture SQL

Un pipeline SQL complet pour Snowflake est fourni dans `snowflake_pipeline.sql`. Il exploite le type natif **VECTOR(FLOAT, 256)** de Snowflake et la fonction **VECTOR_COSINE_SIMILARITY** pour la recherche.

TABLE 14 – Tables principales du pipeline Snowflake

Table	Description
AUDIO_EMBEDDINGS	Embeddings audio 256D + metadonnees (audio_id, filepath)
TEXT_EMBEDDINGS	Embeddings texte 256D + contenu des chunks
AUDIO_QUERIES	Requetes audio de test
RETRIEVAL_RESULTS	Resultats de recherche avec scores et rangs

```

1 CREATE TABLE S2R_DB.S2R_SCHEMA.TEXT_EMBEDDINGS (
2     chunk_id      VARCHAR(255) NOT NULL PRIMARY KEY,
3     document_id   VARCHAR(255),
4     text_content  TEXT,
5     embedding     VECTOR(FLOAT, 256),          -- type natif
6     loaded_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP()
7 );

```

Listing 11 – Schema de la table TEXT_EMBEDDINGS dans Snowflake

7.2 Recherche par similarite vectorielle

```

1 SELECT
2     t.chunk_id,
3     t.document_id,
4     LEFT(t.text_content, 300) AS text_preview,
5     VECTOR_COSINE_SIMILARITY(t.embedding, q.embedding) AS score
6 FROM S2R_DB.S2R_SCHEMA.TEXT_EMBEDDINGS t
7 CROSS JOIN (
8     SELECT embedding FROM S2R_DB.S2R_SCHEMA.AUDIO_QUERIES
9     WHERE audio_id = 'spontaneous-speech-fr-20300'
10 ) q
11 ORDER BY score DESC
12 LIMIT 5;

```

Listing 12 – Recherche top-5 dans Snowflake

7.3 Procedure stockee d'inference

Une procedure stockee Snowpark integre le modele Python directement dans Snowflake :

```
1 CREATE OR REPLACE PROCEDURE S2R_DB.S2R_SCHEMA.S2R_SEARCH(  
2     AUDIO_PATH VARCHAR,  
3     K INTEGER  
4 )  
5 RETURNS TABLE(chunk_id VARCHAR, score FLOAT, text_preview  
6     VARCHAR)  
7 LANGUAGE PYTHON  
8 RUNTIME_VERSION = '3.10'  
9 PACKAGES = ('torch', 'transformers', 'soundfile', 'numpy',  
10    'faiss-cpu')  
11 HANDLER = 's2r_search'  
12 AS $$  
13 def s2r_search(session, audio_path, k):  
14     import numpy as np  
15     from src.speech_encoder import speech_to_embedding  
16     emb = speech_to_embedding(audio_path)  
17     # ... recherche dans la table TEXT_EMBEDDINGS  
18 $$;
```

Listing 13 – Procedure d'inference S2R dans Snowflake

8. Problemes techniques et solutions

Cette section documente les problemes rencontres pendant le developpement et les corrections apportees.

TABLE 15 – Problemes techniques et corrections

Probleme	Cause	Solution
<code>RuntimeError: libtorchcodec unrecognized arguments: -text-chunks-csv</code>	FFmpeg manquant sur Windows	Remplacement de <code>torchaudio.load</code> par <code>soundfile.read</code>
<code>audio_id column not found</code>	Argument absent de <code>validate_handoff</code>	Suppression de l'argument des appels au script
<code>missing column: document_id</code>	Colonne nommee <code>audio_file</code> dans certains CSVs	Normalisation automatique <code>audio_file</code> → <code>audio_id</code>
Segfault DataLoader (Windows)	Colonne absent de <code>corpus_chunks.csv</code>	Derivation automatique depuis <code>chunk_id</code> (regex)
<code>.bat</code> non reconnu	Multiprocessing incompatible	<code>num_workers=0, pin_memory=False</code>
Temps d'entrainement > 2h/epoque	Caracteres accents (UTF-8) dans <code>cmd.exe</code>	Reecriture du <code>.bat</code> en ASCII pur, suppression de <code>chcp</code>
<code>DualEncoderModel(**con</code> crash	Forward MiniLM a chaque batch	Mode rapide : pre-calcul unique des embeddings texte
	<code>text_input_dim</code> absent du config sauvegarde	Correction de <code>export_config()</code> pour sauvegarder <code>text_input_dim</code>

9. Resultats et evaluation

9.1 Bilan quantitatif global

TABLE 16 – Bilan quantitatif du projet complet

Metrique	Valeur	Detail
Fichiers audio WAV	4 985	16 kHz, mono, normalises
Duree totale audio	≈ 17.6 h	$\approx 63\,600$ secondes
Chunks texte	6 744	512 tokens, overlap 50
Paires audio–document	4 985	Jaccard moyen = 0.073
Paires d’entrainement	3 988	80% du dataset
Dimension audio (Wav2Vec2)	768D	94M parametres, gele
Dimension texte (MiniLM)	384D	22M parametres, gele
Espace partage	256D	Dual Encoder projections
Parametres entrainables	~ 300 K	2 couches Linear uniquement
Temps entrainement CPU	~ 15 min	20 epoques, mode rapide
Recall@5 (objectif)	> 0.30	Apres 20 epoques
Recall@10 (objectif)	> 0.45	Apres 20 epoques
MRR (objectif)	> 0.25	Apres 20 epoques

9.2 Tests systeme

TABLE 17 – Resultats des tests systeme

Test	Resultat	Critere
Output shape SpeechEncoder = (768,)	✓	<code>emb.shape == (768,)</code>
Norme L2 ≈ 1.0	✓	$\ \hat{e}\ _2 = 1.0000$
Determinisme du SpeechEncoder	✓	mime input $\rightarrow$ mime output
Batch encoding (N , 768)	✓	shape [4985, 768]
Validation handoff P1/P2	✓	manifest + paires OK
Pre-calcul MiniLM (N , 384)	✓	shape (6744, 384)
Export embeddings (N , 256)	✓	shape (6744, 256)
API /health	✓	<code>"status": "ok"</code>
API /search/audio	✓	5 resultats avec scores
Interface Gradio	✓	Demarrage + 2 onglets OK

10. Conclusion et perspectives

10.1 Bilan

Ce projet realise un systeme complet de **Speech-to-Retrieval** : depuis la collecte et le nettoyage des donnees audio et texte (P1), l'encodage vocal sans ASR via Wav2Vec2 (P2), l'alignement des espaces multimodaux par Dual Encoder contrastif (P3), jusqu'a l'exposition via API REST et interface web interactive (P4).

L'innovation principale est le **mode rapide d'entrainement** : en pre-calculant les embeddings texte une seule fois, le pipeline rend l'entrainement accessible sur CPU ordinaire (~15 minutes pour 20 epoques), ce qui facilite l'experimentation et l'iteration rapide.

10.2 Perspectives d'amelioration

- **Modele multilingue** — Remplacer Wav2Vec2-base par facebook/wav2vec2-xls-r-300m (300M parametres, 128 langues) pour un support naturel du francais et du darija
- **Fine-tuning partiel** — Degeler les 2 a 4 dernieres couches Transformer de Wav2Vec2 lors de l'entrainement pour une meilleure adaptation semantique (necessite GPU)
- **Corpus elargi** — Passer de 570 a 1 500 articles Wikipedia ($\approx 18\,000$ chunks) pour couvrir davantage de domaines
- **Hard negative mining** — Construire des negatifs difficiles (documents semantiquement proches mais non pertinents) pour ameliorer la discrimination du modele
- **Encoder texte plus puissant** — Remplacer MiniLM (384D) par all-mpnet-base-v2 (768D) pour des representations textuelles plus riches
- **Augmentation audio** — Ajout de bruit de fond, pitch shifting, et changement de debit pour la robustesse
- **Deploiement Snowflake** — Exploiter le pipeline SQL genere pour une integration en production dans un data warehouse d'entreprise

11. Annexe : Structure du projet

```

1 inpt_audioprocessing/
2 |-- src/
3 |   '-- speech_encoder.py           # SpeechEncoder (Wav2Vec2),
   |   _load_wav()
4 |-- training/
5 |   |-- models.py                   # DualEncoderModel (audio +
   |   text projections)
6 |   |-- losses.py                   # contrastive_loss (InfoNCE),
   |   triplet_loss

```

```

7 |   |-- metrics.py           # Recall@5, Recall@10, MRR
8 |   |-- data.py             # Dataset, TextEmbeddingStore,
   fast_collate
9 |   |-- train_dual_encoder.py # Script d'entraînement
   principal
10 |   |-- precompute_text.py   # Pre-calcul embeddings MiniLM
11 |   |-- export_text_embeddings.py # Export embeddings 256D
   post-training
12 |   '-- validate_handoff.py   # Validation des donnees P1/P2
13 |-- api/
14 |   '-- main.py             # API FastAPI (5 endpoints)
15 |-- demo/
16 |   '-- app.py              # Interface Gradio (2 onglets)
17 |-- notebooks/
18 |   |-- P1_Data_Engineer_v3.ipynb
19 |   |-- P2_Speech_AI_Engineer_v2_FIXED.ipynb
20 |   '-- S2R_Colab_Kaggle.ipynb # Notebook cloud training
21 |-- embeddings/           # Matrices numpy + manifests
   CSV
22 |-- models/               # Checkpoints entraines
23 |-- data/                 # Audio + texte + paires
24 |-- inference.py          # CLI end-to-end
25 |-- run_pipeline.bat      # Pipeline Windows
26 |-- run_pipeline.sh       # Pipeline Linux/macOS
27 |-- snowflake_pipeline.sql # Pipeline Snowflake complet
28 '-- requirements.txt

```

Listing 14 – Arborescence du projet

12. References

- [1] Baevski et al. (2020). *wav2vec 2.0 : A Framework for Self-Supervised Learning of Speech Representations*. NeurIPS 2020.
- [2] Conneau et al. (2021). *Unsupervised Cross-lingual Representation Learning for Speech Recognition*. Interspeech 2021.
- [3] Reimers & Gurevych (2019). *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
- [4] Johnson et al. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data.
- [5] Panayotov et al. (2015). *LibriSpeech : an ASR corpus based on public domain audio books*. ICASSP 2015.
- [6] Chen et al. (2020). *A Simple Framework for Contrastive Learning of Visual Representations (SimCLR)*. ICML 2020.
- [7] Radford et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision (CLIP)*. ICML 2021.
- [8] HuggingFace. *openslr/librispeech_asr*. https://huggingface.co/datasets/openslr/librispeech_asr

- [9] HuggingFace. *facebook/wav2vec2-base-960h*. <https://huggingface.co/facebook/wav2vec2-base-960h>
- [10] HuggingFace. *sentence-transformers/all-MiniLM-L6-v2*. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [11] FAISS. *Faiss : A library for efficient similarity search*. <https://github.com/facebookresearch/faiss>
- [12] Snowflake Inc. *Snowflake VECTOR type and similarity functions*. Snowflake Documentation, 2024.